

tmux documentation

§03 — Runtime Isolation on Termux

Revision: 1 **Last modified:** 2026-05-22T07:20:02Z **Authority:** vasic-digital tmux project (research-only) **Maintainer:** milosvasic **Scope:** What replaces `systemd-run --user --scope` on Termux — the central engineering question for the port

1. The problem

Today the tmx wrapper enforces per-session resource isolation with two backends, dispatched on `uname -s`:

Host	Mechanism	Caps enforced
Linux	<code>systemd-run --user --scope --unit=tmx-NAME.scope -p MemoryMax=... -p CPUQuota=... -p TasksMax=...</code>	RAM (MemoryMax), CPU% (CPUQuota), task count (TasksMax), kernel-enforced via cgroup-v2 controllers
macOS	<code>tmx-rlimit-wrapper.sh</code> setting <code>RLIMIT_CPU + RLIMIT_NPROC</code>	CPU seconds + per-user process count, kernel-enforced via XNU's POSIX <code>setrlimit</code> . Honest gap: <code>RLIMIT_AS</code> is NOT enforced for unprivileged processes on XNU.

Neither mechanism is available on Termux.

- `systemd-run`: there is no `systemd` on Android. There is no `--user` instance. The user installs Termux, gets `bash`, gets no init manager. (Sources: Termux discussion #17585; LWN Termux article — see §01 sources.)
- macOS branch: relies on `tmx-rlimit-wrapper.sh` which is a Mach-O binary. We can rebuild it for Android-arm64; the `rlimit` syscalls themselves work (see §2). But the WRAPPER as compiled is macOS-only.

2. What CAN we use on Termux

Four candidates, in order of likely viability:

2.1 `setrlimit(2)` directly — RECOMMENDED PRIMARY

The kernel under Android is Linux. The bionic libc exposes `setrlimit / getrlimit` and `prlimit` (the latter on 64-bit only — see Termux's `util-linux/build.sh` which gates the `prlimit` feature on 64-bit). Source: <https://android.googlesource.com/platform/bionic/+HEAD/docs/status.md> + <https://github.com/termux/termux-packages/blob/master/packages/util-linux/build.sh>.

The bionic source tree carries architecture-specific shim files (`libc/arch-arm/syscalls/setrlimit.S`, `libc/arch-mips/syscalls/setrlimit.S`, etc.) confirming first-class support since the earliest Android revisions.

Confirmed-portable resource limits on Termux:

Limit	Linux semantics	Termux availability
RLIMIT_AS (max virtual address space)	enforced by kernel	enforced — Linux kernel underneath, same <code>do_anonymous_page / mmap</code> checks
RLIMIT_CPU (CPU seconds; SIGXCPU → SIGKILL)	enforced by kernel	enforced — kernel timer interrupt path is identical
RLIMIT_NPROC (per-real-UID process count)	enforced by kernel at <code>fork(2)</code>	enforced for the Termux UID — but see §4 caveat below
RLIMIT_NOFILE (open file descriptors)	enforced	enforced
RLIMIT_CORE (core file size)	enforced	enforced but core dumps usually disabled on Android
RLIMIT_FSIZE (max file size)	enforced	enforced
RLIMIT_STACK (stack size)	enforced	enforced

Compared to macOS's gap (RLIMIT_AS not honoured by XNU), Termux is STRICTLY BETTER for our needs: we actually get memory address-space ceilings the kernel enforces. This is a small win for the port.

The macOS `tmx-rlimit-wrapper.sh` design ports directly: the script calls `ulimit -v $MEM_KB; ulimit -t $CPU_SEC; ulimit -u $PROC_MAX; exec "$@"`. Termux's bash supports `ulimit -v` (RLIMIT_AS), `-t` (RLIMIT_CPU), `-u` (RLIMIT_NPROC). No code changes beyond the OS detection.

2.2 prlimit(1) via the util-linux package

From `pkg install util-linux` (on 64-bit Termux). `prlimit` lets us SET limits on an already-running process — useful if we want to defer cap application until after the tmux server forks. The macOS wrapper sets limits BEFORE `exec` so we don't need this, but it's good to know `prlimit` is on the table.

Source: <https://github.com/termux/termux-packages/blob/master/packages/util-linux/build.sh> — line: `ac_cv_func_prlimit=yes` set in 64-bit builds.

2.3 cgroup-v2 (rooted devices only) — OPTIONAL POWER-USER PATH

If the operator has a rooted device (Magisk / KernelSU), they can run `su -c 'mkdir /sys/fs/cgroup/tmx-NAME; echo $PID > /sys/fs/cgroup/tmx-NAME/cgroup.procs'` from within Termux. Some custom kernels also expose a writable cgroup hierarchy without root.

This path is NOT recommended as a default because:

- Most Termux users are unrooted by design (security posture).
- Rooting voids warranty and many SafetyNet-protected apps refuse to run.
- Kernel cgroup-v2 support is patchy on older Android versions (see <https://github.com/ravindu644/Droidspaces-OSS>: "kernel namespaces (PID, mount, network, IPC, UTS) and cgroups to create fully isolated environments" — works on Linux kernel 3.18+ but availability depends on which controllers the OEM compiled in).

If we add a Termux branch to `tmx.template`, detecting "rooted + cgroup-v2 writable" and using it would give us feature parity with the Linux branch. The detector is `[ -w /sys/fs/cgroup/cgroup.controllers ]`.

2.4 SELinux deny + Android lmkd

Not a replacement — these are constraints we must respect:

- SELinux blocks unprivileged Termux from setting OOM scores below 0 on most policy variants. `/proc/$pid/oom_score_adj` is writable but lmkd (Android's userspace low-memory killer) maintains its own table indexed by Android UID. If lmkd decides Termux is "background, low priority" because the screen is off, it can `SIGKILL` the tmux server regardless of our `oom_score_adj=-500` write.
- **Phantom Process Killer (Android 12+)**. Limits child-process count per app to 32 by default. tmux spawning many panes can trip this. Disabling via developer-options ADB toggle is the documented workaround. See <https://maheshtechnical.com/fix-termux-error-process-completed-signal-9-disable-phantom-process-killer-in-android-12-13/> and the upstream Termux issue <https://github.com/termux/termux-app/issues/2366>.

We CANNOT defeat lmkd or the phantom killer from userspace. The honest answer is: SKIP-with-reason in any test that asserts "the server survives N hours of background time," and document the wake-lock + battery-optimization workaround for operators (see §05).

3. Honest-gap citation table (§11.4.81)

Per §11.4.81 (cross-platform-parity mandate), every isolation primitive used in the wrapper needs an explicit per-OS row. Here is the proposed table for `docs/guide/README.md` §5.6 when Termux lands:

Primitive	Linux	macOS (Darwin)	Termux (Android, unrooted)
Memory cap	cgroup-v2 MemoryMax (hard, kernel-enforced)	XNU does NOT enforce <code>RLIMIT_AS</code> for unprivileged processes — honest gap, no fallback	<code>RLIMIT_AS</code> via <code>ulimit -v</code> — enforced by Linux kernel underneath Android
CPU cap	cgroup-v2 CPUQuota (e.g. 200% = 2 cores)	<code>RLIMIT_CPU</code> (cumulative CPU seconds → <code>SIGXCPU</code> then <code>SIGKILL</code>)	<code>RLIMIT_CPU</code> (same as macOS)
Task cap	cgroup-v2 TasksMax (per-cgroup process count)	<code>RLIMIT_NPROC</code> (per-real-UID, applies across the whole user session)	<code>RLIMIT_NPROC</code> (same as macOS) + 32-child cap from Android 12 Phantom Process Killer unless user disables it via ADB
OOM priority	<code>oom_score_adj</code> + cgroup-controlled OOM	no equivalent (XNU has no OOM killer at all)	<code>oom_score_adj</code> writable but lmkd may override based on app priority Doze + Phantom Process Killer may freeze/kill unless termux-wake-lock held AND battery optimization disabled
Background survival	systemd keeps service running	macOS keeps the tmux server running until logout	

The honest-gap row vs macOS: Termux is **strictly better** on memory (we get a cap; macOS gets none) and **strictly worse** on background survival (macOS doesn't freeze; Termux does). CPU and task caps are functionally tied.

4. RLIMIT_NPROC sharing — important caveat

RLIMIT_NPROC is per-real-UID on Linux. On Termux, the real UID is the Termux app's UID. Every tmx session shares the same UID — they ALL count against the same RLIMIT_NPROC. Setting RLIMIT_NPROC=4096 in one wrapper doesn't isolate session A from session B; it caps the total count of processes the Termux app can spawn.

On Linux's cgroup-based approach, TasksMax is per-cgroup — true per-session isolation. We lose that on Termux.

Practical impact: if a session goes fork-bomby, OTHER sessions also start failing `fork()` with EAGAIN. The tmux servers themselves shouldn't crash (they just see fewer fork-able resources) but the user experience for the other sessions degrades. Document in operator guide.

Same caveat applies to macOS today — RLIMIT_NPROC is also per-real-UID on Darwin. So Termux inherits the existing macOS limitation rather than introducing a new one. Honest framing per §11.4.6.

5. The proposed Termux branch in `tmx.template`

Sketch (NOT to be implemented in this research mission — just for design clarity):

```
case "$HOST_OS" in
  Linux)
    if [ -n "${TERMUX_VERSION:-}" ]; then
      # Termux on Android — Linux uname but no systemd, no
      # cgroups.
      # Use rlimit wrapper, same as macOS.
      "$RLIMIT_WRAPPER" "$MEM_KB" "$CPU_SEC" "$PROC_MAX" \
        "$TMUX_BIN" -L "$SOCK_LABEL" -f "$TMUX_CONF" new-
      session ...
      TMX_CLASSIFICATION="tmx-supported-termux"
    elif [ "$_scope_ok" -eq 1 ]; then
      # Real Linux with systemd-user — existing path.
      systemd-run --user --scope ...
      TMX_CLASSIFICATION="tmx-supported"
    else
      TMX_CLASSIFICATION="tmx-degraded"
      # ...
    fi
  ;;
  Darwin)
    # Existing macOS path.
    "$RLIMIT_WRAPPER" "$MEM_KB" "$CPU_SEC" "$PROC_MAX"
    "$TMUX_BIN" ...
  ;;
esac
```

```
;;  
esac
```

The `$TERMUX_VERSION` env var is exported by Termux's login script (sourced in `~/ .bashrc` equivalent at session start) and is the canonical "am I running inside Termux?" detector. Source: <https://github.com/termux/termux-packages/wiki/Termux-environment-variables>.

6. The wake-lock requirement

Setting `rlimit` and starting `tmux` is not enough. For long-running sessions (the whole point of `tmux`) Android's Doze mode WILL throttle CPU and network when the screen is off. Source: <https://github.com/termux/termux-app/issues/377>.

The standard mitigation is `termux-wake-lock`, a command shipped by Termux that acquires an Android partial wake lock for the Termux process group. This keeps the CPU available even with screen off.

Two design options for the wrapper:

1. **Always-on:** `tmx new` calls `termux-wake-lock` first, `tmx kill-server` calls `termux-wake-unlock`. Simple, predictable, costs battery.
2. **Opt-in:** wrapper detects Termux and PRINTS a notice "to keep this session alive in background, run `termux-wake-lock`". Operator chooses.

Recommend option 2 for default behaviour with a `--background` flag that triggers option 1 explicitly. This respects user intent (some operators DO want the session to suspend with the device).

7. Summary

We can build a viable Termux isolation layer using `setrlimit + ulimit`. The result has the same shape as our macOS branch with two additions:

- Memory address-space cap actually works (better than macOS).
- Background survival is fragile (worse than macOS).

A rooted-cgroup-v2 sub-path is feasible as an opt-in for power users, but not the default.

The §11.4.81 honest-gap citation has THREE rows now, and Termux gets its own column.

Sources

- <https://android.googlesource.com/platform/bionic/+/HEAD/docs/status.md> — bionic API support (`setrlimit`, `prlimit`, `fork`, `signals`)
- <https://github.com/termux/termux-packages/blob/master/packages/util-linux/build.sh> — `prlimit` gate on 64-bit
- <https://man7.org/linux/man-pages/man2/getrlimit.2.html> — Linux `rlimit` semantics (kernel-enforced under Android too)
- <https://source.android.com/docs/core/perf/lmkd> — Android low-memory killer daemon
- <https://github.com/termux/termux-app/issues/377> — Doze affects `tmux` in Termux
- <https://github.com/termux/termux-app/issues/2366> — Phantom Process Killer signal-9 kills since Android 12

- <https://maheshtechicals.com/fix-termux-error-process-completed-signal-9-disable-phantom-process-killer-in-android-12-13/> — ADB-driven workaround
- <https://dev.to/elenbit/unreliable-linux-containers-on-android-addressing-integration-networking-and-stability-for-2p5b> — confirmation cgroups not writable from unprivileged userspace
- <https://github.com/ravindu644/Droidspaces-OSS> — example of rooted-kernel cgroup approach